WPF: привязки данных и архитектура MVVM

1. Паттерн MVVM

В предыдущей лекции мы разобрались как делать XAML-разметку и создавать формы с нужным нам внешним видом, но они не являются функциональными: кнопки ничего не делают, поля не связаны с данными. Чтобы приложение заработало, нужно написать логику. Вопрос в том, где именно эту логику разместить. Для решения этой проблемы применяют паттерн **MVVM**.

Бизнес-логика в code-behind

Самый очевидный путь — написать всё прямо в code-behind, в обработчиках событий. Нажата кнопка «Сохранить» — пишем логику в `Button_Click`:

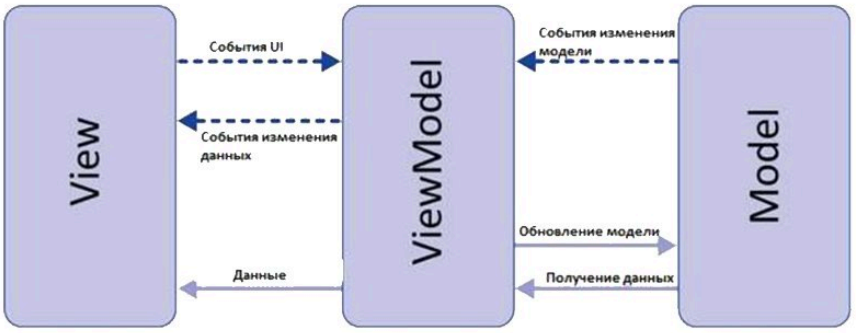
```
private async void SaveButton_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrWhiteSpace(TitleBox.Text))
    {
        ErrorLabel.Visibility = Visibility.Visible;
        return;
    }

    SaveButton.IsEnabled = false;
    LoadingBar.Visibility = Visibility.Visible;

    await _bookService.SaveAsync(new Book { Title = TitleBox.Text });

    LoadingBar.Visibility = Visibility.Collapsed;
    SaveButton.IsEnabled = true;
}
```

Код работает, но у него есть проблемы.

- **Логика намертво привязана к элементам интерфейса.** Валидация знает про `TitleBox`, сохранение знает про `LoadingBar` и `SaveButton`. Если переименовать поле или перенести его в другой компонент — код сломается.
- **Код нельзя протестировать.** Чтобы проверить валидацию, нужно создать реальное окно с реальными элементами. Unit-тест здесь невозможен.
- **Код нельзя переиспользовать.** Если понадобится такая же форма в другом месте приложения — логику придётся копировать.

MVVM

MVVM (**Model — View — ViewModel**) — это архитектурный паттерн: соглашение о том, как организовать код.



Model — данные и бизнес-логика. Классы сущностей (`Book` , `Author`), репозитории, сервисы. Не импортирует ничего из WPF и не знает о существовании интерфейса.

View — XAML-разметка и минимальный code-behind. Отвечает только за отображение: как выглядит кнопка, где расположено поле. Не содержит логики — только привязки к ViewModel.

ViewModel — посредник между View и Model. Хранит состояние экрана (какие данные сейчас отображаются, идёт ли загрузка, есть ли ошибка), содержит команды (что произойдёт при нажатии кнопки), вызывает методы сервисов. Не знает о конкретных элементах интерфейса.

Правила взаимодействия слоёв

Слои взаимодействуют строго в одном направлении:

- View знает о ViewModel — подписывается на её свойства через привязки
- ViewModel знает о Model — вызывает методы сервисов и репозиториев
- Model не знает ни о ViewModel, ни о View
- ViewModel не знает о View — не обращается к элементам интерфейса напрямую

Это ключевое правило. ViewModel не должна содержать ни одной строки вроде `TitleBox.Text` или `SaveButton.IsEnabled` . Она управляет только свойствами — а View сам решает, как их отобразить.

Почему ViewModel не должен знать о View

- **Тестируемость.** ViewModel — обычный C#-класс без зависимости от WPF. Его можно создать в тесте, вызвать команду и проверить результат:

```
// Unit-тест — никакого окна, никакого UI
var vm = new BookEditViewModel(new FakeBookService());
vm.Title  = "Мастер и Маргарита";
vm.Author = "Булгаков";

await vm.SaveCommand.ExecuteAsync();

Assert.True(vm.IsSaved);
```

- **Переиспользование.** Одну ViewModel можно подключить к разным View. Например, одна `BookEditViewModel` может работать и с полной формой редактирования, и с компактной панелью быстрого редактирования.
- **Независимая разработка.** Дизайнер может менять разметку View, разработчик — логику ViewModel. Пока интерфейс свойств и команд не меняется, они не мешают друг другу.

Структура папок проекта

В лекции 8 мы создали папки `Views` и `Resources`. Теперь добавим остальные:

```
LibraryApp/
├── Models/          — модели данных
│   └── Book.cs
│
├── ViewModels/      — логика представления
│   ├── ViewModelBase.cs
│   └── BookEditViewModel.cs
│
├── Views/           — XAML-разметка и code-behind
│   └── MainWindow.xaml
│
├── Services/        — бизнес-логика и доступ к данным
│   ├── IBookService.cs
│   └── BookService.cs
│
├── Resources/       — стили и ресурсы
│
├── App.xaml
├── App.xaml.cs
└── LibraryApp.csproj
```

Models содержат данные и бизнес-правила. Не зависят от WPF.

ViewModels содержат логику представления: состояние экрана, команды, обращения к сервисам. Не зависят от конкретных элементов интерфейса.

Views содержат только разметку и минимальный code-behind. Вся логика делегирована во ViewModel.

Services содержат работу с базой данных, файлами и внешними API.

2. ViewModel и уведомления об изменениях

Чтобы привязки данных работали в обе стороны, UI должен знать, когда свойство ViewModel изменилось. Обычное C#-свойство это не умеет — привязка прочитает значение один раз при инициализации и больше не будет его обновлять. Нужен механизм уведомлений.

INotifyPropertyChanged

Чтобы UI реагировал на изменения свойств, объект должен реализовать интерфейс `INotifyPropertyChanged`:

```
public interface INotifyPropertyChanged
{
    event PropertyChangedEventHandler? PropertyChanged;
}
```

Когда свойство меняется, объект вызывает событие `PropertyChanged` с именем свойства. Система привязок WPF подписана на это событие и обновляет UI при его получении.

В минимальной реализации каждое свойство выглядит так:

```
public class BookEditViewModel : INotifyPropertyChanged
{
    private string _title = "";

    public string Title
    {
        get => _title;
        set
        {
            if (_title == value) return;
            _title = value;
            PropertyChanged?.Invoke(this,
                new PropertyChangedEventArgs(nameof(Title)));
        }
    }

    public event PropertyChangedEventHandler? PropertyChanged;
}
```

Этот шаблонный код повторяется для каждого свойства. В реальных проектах его не пишут вручную, а используют готовую библиотеку.

CommunityToolkit.Mvvm

`CommunityToolkit.Mvvm` — официальная библиотека Microsoft для MVVM в .NET-приложениях. Она берёт на себя весь шаблонный код уведомлений и команд. Устанавливается через NuGet:

```
dotnet add package CommunityToolkit.Mvvm
```

Или через интерфейс Visual Studio в менеджере NuGet-пакетов.

Библиотека использует **source generators** — механизм компилятора C#, который на этапе сборки автоматически дописывает код по атрибутам. Мы пишем атрибут на поле, компилятор генерирует полноценное свойство с уведомлением.

ObservableObject — базовый класс

Вместо ручной реализации оповещений, унаследуем класс от `ObservableObject` из библиотеки:

```
using CommunityToolkit.Mvvm.ComponentModel;

public partial class BookEditViewModel : ObservableObject
{
    // свойства и команды — ниже
}
```

Класс обязательно должен быть объявлен как `partial` — это требование source generators: они дописывают сгенерированный код в тот же класс через механизм разделённых классов.

[ObservableProperty] — свойства с уведомлением

Атрибут `[ObservableProperty]` на приватном поле генерирует полноценное публичное свойство с уведомлением:

```
public partial class BookEditViewModel : ObservableObject
{
    [ObservableProperty]
    private string _title = "";

    [ObservableProperty]
    private string _author = "";

    [ObservableProperty]
```

```
    private int _year = DateTime.Now.Year;
}
```

Source generator читает эти поля при сборке и генерирует для каждого из них свойство. Для `_title` получится примерно:

```
// Сгенерированный код (упрощённо)
public string Title
{
    get => _title;
    set
    {
        if (_title == value) return;
        _title = value;
        OnPropertyChanged(nameof(Title));
    }
}
```

Соглашение об именовании: поле называется `_camelCase`, сгенерированное свойство — `PascalCase`. `_title` → `Title`, `_isAvailable` → `IsAvailable`, `_isSaving` → `IsSaving`. Если поле не начинается с `_` — source generator не сработает.

Вычисляемые свойства

Некоторые свойства не хранят значение, а вычисляются на основе других. Например, кнопка «Сохранить» должна быть активна только когда заполнены обязательные поля и не идёт сохранение:

```
public bool CanSave =>
    !string.IsNullOrWhiteSpace(Title) &&
    !string.IsNullOrWhiteSpace(Author) &&
    !IsSaving;
```

Проблема та же, что и раньше: когда меняется `Title` или `Author`, UI не знает что `CanSave` тоже изменился. Атрибут `[NotifyPropertyChangedFor]` решает это декларативно — он указывает какие дополнительные свойства нужно уведомить при изменении поля:

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(CanSave))]
private string _title = "";

[ObservableProperty]
[NotifyPropertyChangedFor(nameof(CanSave))]
private string _author = "";
```

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(CanSave))]
private bool _isSaving;
```

Теперь при каждом изменении `_title`, `_author` или `_isSaving` — UI получит уведомление и для самого свойства, и для `CanSave`.

Полная ViewModel

Соберём всё вместе:

```
using CommunityToolkit.Mvvm.ComponentModel;

public partial class BookEditViewModel : ObservableObject
{
    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(CanSave))]
    private string _title = "";

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(CanSave))]
    private string _author = "";

    [ObservableProperty]
    private int _year = DateTime.Now.Year;

    [ObservableProperty]
    private string _genre = "";

    [ObservableProperty]
    private bool _isAvailable = true;

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(CanSave))]
    private bool _isSaving;

    public bool CanSave =>
        !string.IsNullOrWhiteSpace(Title) &&
        !string.IsNullOrWhiteSpace(Author) &&
        !IsSaving;
}
```

Установка DataContext

ViewModel передаётся во View через свойство `DataContext`. Когда `DataContext` окна установлен в экземпляр ViewModel, все привязки внутри окна автоматически обращаются к её свойствам.

`DataContext` наследуется по дереву элементов: достаточно установить его один раз на корневом элементе — все вложенные `Grid`, `StackPanel`, `TextBlock` будут использовать тот же объект.

Удобнее всего устанавливать `DataContext` главного окна в `App.xaml.cs`. Сначала уберём `StartupUri` из `App.xaml`, так как окно теперь создаётся вручную:

```
<!-- App.xaml — убираем StartupUri -->
<Application x:Class="LibraryApp.App"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Application.Resources />
</Application>
```

```
// App.xaml.cs
public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);

        var window = new MainWindow
        {
            DataContext = new MainWindowViewModel()
        };

        window.Show();
    }
}
```

Для дочерних View — например формы редактирования книги — `DataContext` устанавливается в момент создания View, обычно из окна:

```
var view = new BookEditView
{
    DataContext = new BookEditViewModel()
};
view.ShowDialog();
```

3. Привязки данных

ViewModel готова — она хранит состояние и умеет уведомлять об изменениях. Теперь нужно связать её с интерфейсом. Привязка данных автоматически синхронизирует свойства ViewModel со свойствами элементов управления — в одну или в обе стороны.

Зачем нужны привязки

Без привязок синхронизация UI и данных — ручная работа. Загрузить данные в форму:

```
TitleBox.Text          = book.Title;
AuthorBox.Text         = book.Author;
YearBox.Text           = book.Year.ToString();
AvailableBox.IsChecked = book.IsAvailable;
```

Сохранить обратно:

```
book.Title       = TitleBox.Text;
book.Author      = AuthorBox.Text;
book.Year        = int.Parse(YearBox.Text);
book.IsAvailable = AvailableBox.IsChecked ?? false;
```

Добавилось поле — нужно не забыть дописать в обоих местах. Данные обновились в фоне — нужно снова вручную обновить все элементы. Привязки решают эту проблему: синхронизация происходит автоматически.

Синтаксис привязки

Привязка записывается в XAML как `{Binding ...}`:

```
<!-- Привязка к свойству DataContext -->
<TextBlock Text="{Binding Title}" />

<!-- Привязка к вложенному свойству -->
<TextBlock Text="{Binding Publisher.Name}" />

<!-- Привязка к самому DataContext целиком -->
<TextBlock Text="{Binding}" />
```

При привязке к вложенному свойству WPF проходит по цепочке. Если любое звено равно `null` — привязка возвращает `null` без исключения.

Режимы привязки

Режим определяет направление передачи данных:

```
<!-- OneWay: ViewModel → UI. По умолчанию для TextBlock -->
<TextBlock Text="{Binding Title}" />

<!-- TwoWay: ViewModel ⇔ UI. По умолчанию для TextBox -->
<TextBox Text="{Binding Title}" />

<!-- OneTime: читается один раз при инициализации, удобно для статичных
данных -->
<TextBlock Text="{Binding Id, Mode=OneTime}" />
```

На практике умолчания работают правильно для большинства случаев — явно указывать `Mode` нужно редко.

UpdateSourceTrigger

Определяет когда изменение в UI передаётся обратно в ViewModel. По умолчанию для `TextBox` это `LostFocus` — ViewModel узнаёт об изменении только когда поле теряет фокус.

Чаще нужно `PropertyChanged` — обновление на каждый введённый символ:

```
<TextBox Text="{Binding Title, UpdateSourceTrigger=PropertyChanged}" />
```

Это важно для `CanSave` — без `PropertyChanged` кнопка «Сохранить» не будет реагировать на ввод пока поле не потеряет фокус.

FallbackValue и TargetNullValue

```
<!-- TargetNullValue — значение когда свойство равно null -->
<TextBlock Text="{Binding Genre, TargetNullValue='Не указан'}" />

<!-- FallbackValue — значение когда привязка не смогла получить данные -->
<TextBlock Text="{Binding Title, FallbackValue='Загрузка...'}" />
```

`TargetNullValue` — когда данные есть, но значение может быть `null`. `FallbackValue` — защита от ошибок привязки, удобна при разработке.

StringFormat — форматирование в привязке

Числа и даты можно форматировать прямо в привязке без конвертеров:

```
<TextBlock Text="{Binding Year,      StringFormat='Год издания: {0}'}" />
<TextBlock Text="{Binding Price,     StringFormat='{0:N2} ₽'}" />
<TextBlock Text="{Binding CreatedAt, StringFormat='Добавлена:
{0:dd.MM.yyyy}'}" />
```

Если строка формата начинается с `{` — оборачиваем в одинарные кавычки, иначе парсер XAML воспримет фигурную скобку как начало разметки.

Value Converter

Иногда тип данных в ViewModel не совпадает с тем, что ожидает элемент управления. Например, `ProgressBar` должен быть видим или скрыт в зависимости от булевого `IsSaving`, а элементы WPF принимают `Visibility`, а не `bool`. Конвертер реализует интерфейс `IValueConverter`:

```
// Converters/BoolToVisibilityConverter.cs
public class BoolToVisibilityConverter : IValueConverter
{
    public object Convert(
        object value, Type targetType,
        object parameter, CultureInfo culture)
        => value is true ? Visibility.Visible : Visibility.Collapsed;

    public object ConvertBack(
        object value, Type targetType,
        object parameter, CultureInfo culture)
        => value is Visibility.Visible;
}
```

Регистрируем как ресурс в `App.xaml` — для этого нужно подключить пространство имён `Converters`:

```
<Application
    ...
    xmlns:conv="clr-namespace:LibraryApp.Converters">
    <Application.Resources>
        <conv:BoolToVisibilityConverter x:Key="BoolToVisibility" />
    </Application.Resources>
</Application>
```

Использование:

```
<ProgressBar Visibility="{Binding IsSaving,
             Converter={StaticResource BoolToVisibility}}" />
```

WPF поставляет встроенный `BooleanToVisibilityConverter` — для этого конкретного случая писать свой не нужно:

```
<Window.Resources>
    <BooleanToVisibilityConverter x:Key="BoolToVis" />
</Window.Resources>
```

DataTrigger

`DataTrigger` — триггер, который реагирует на значение привязки. В отличие от обычного `Trigger` из лекции 8 (который следит за свойством самого элемента), `DataTrigger` следит за данными во ViewModel.

Типичный пример — изменить цвет текста в зависимости от статуса книги:

```
<Style x:Key="StatusText" TargetType="TextBlock">
    <Setter Property="Foreground" Value="Black" />
    <Style.Triggers>

        <DataTrigger Binding="{Binding IsAvailable}" Value="True">
            <Setter Property="Foreground" Value="#16A34A" />
            <Setter Property="FontWeight"  Value="SemiBold" />
        </DataTrigger>

        <DataTrigger Binding="{Binding IsAvailable}" Value="False">
            <Setter Property="Foreground" Value="#DC2626" />
        </DataTrigger>

    </Style.Triggers>
</Style>
```

```
<TextBlock Text="{Binding StatusText}"
           Style="{StaticResource StatusText}" />
```

Когда `IsAvailable` во ViewModel меняется — текст автоматически меняет цвет. Никакого кода в code-behind.

`MultiDataTrigger` срабатывает когда выполняются несколько условий по привязкам одновременно:

```
<Style.Triggers>
    <MultiDataTrigger>
```

```
        <MultiDataTrigger.Conditions>
            <Condition Binding="{Binding IsAvailable}" Value="True" />
            <Condition Binding="{Binding IsSaving}"    Value="False" />
        </MultiDataTrigger.Conditions>
        <Setter Property="Foreground" Value="#16A34A" />
    </MultiDataTrigger>
</Style.Triggers>
```

Подключаем привязки к форме

Возьмём `BookEditViewModel` из раздела 2 и подключим её к форме из лекции 8.

Поля формы получают привязки с `UpdateSourceTrigger=PropertyChanged` :

```
<!-- Было -->
<TextBox Grid.Row="0" Grid.Column="1" MaxLength="200" />

<!-- Стало -->
<TextBox Grid.Row="0" Grid.Column="1"
         Text="{Binding Title, UpdateSourceTrigger=PropertyChanged}" />
```

`ProgressBar` появляется автоматически когда идёт сохранение:

```
<!-- Было: скрыт навсегда -->
<ProgressBar IsIndeterminate="True" Height="4"
             Visibility="Collapsed" />

<!-- Стало: управляется через ViewModel -->
<ProgressBar IsIndeterminate="True" Height="4"
             Visibility="{Binding IsSaving,
                          Converter={StaticResource BoolToVis}}" />
```

`CheckBox` синхронизируется двусторонне:

```
<CheckBox Content="В наличии"
          IsChecked="{Binding IsAvailable}" />
```

Теперь форма живая: поля обновляют ViewModel на каждый символ, `ProgressBar` появляется и исчезает автоматически.

4. Команды

Привязки синхронизируют данные между ViewModel и UI. Но данные — это состояние, а приложению нужны ещё и действия: сохранить книгу, удалить, открыть форму редактирования. Для этого в WPF существуют команды.

Интерфейс ICommand

Команда в WPF — любой объект, реализующий `ICommand`:

```
public interface ICommand
{
    // Можно ли выполнить команду прямо сейчас?
    bool CanExecute(object? parameter);

    // Выполнить команду
    void Execute(object? parameter);

    // Событие: условие CanExecute изменилось, кнопке нужно обновиться
    event EventHandler? CanExecuteChanged;
}
```

Кнопка с привязкой `Command="{Binding SaveCommand}"` автоматически:

- вызывает `CanExecute` и устанавливает `IsEnabled` в результат
- вызывает `Execute` при нажатии
- обновляет `IsEnabled` при срабатывании `CanExecuteChanged`

[RelayCommand] — команды через атрибут

Так же как `[ObservableProperty]` генерирует свойства, атрибут `[RelayCommand]` генерирует команды. Он ставится на **метод**, который содержит логику действия — source generator создаёт соответствующее свойство-команду.

Соглашение об именовании: метод называется в стиле действия, команда получает суффикс `Command`. Метод `Save()` → команда `SaveCommand`, метод `Delete()` → команда `DeleteCommand`.

Синхронная команда — из обычного метода:

```
using CommunityToolkit.Mvvm.Input;

public partial class BookEditViewModel : ObservableObject
{
    [RelayCommand]
    private void Cancel()
    {
```

```
        Title  = "";
        Author = "";
        Year   = DateTime.Now.Year;
        Genre  = "";
    }
}
```

Асинхронная команда — из `async Task` метода. Тот же атрибут, никакого отдельного класса:

```
[RelayCommand]
private async Task SaveAsync()
{
    IsSaving = true;
    try
    {
        await _bookService.SaveAsync(new Book
        {
            Title  = Title,
            Author = Author,
            Year   = Year,
            Genre  = Genre
        });
    }
    finally
    {
        IsSaving = false;
    }
}
```

Для асинхронного метода generator создаёт команду, которая автоматически блокирует повторный запуск пока выполняется предыдущий — без какого-либо дополнительного кода.

CanExecute — условие доступности

Чтобы команда знала когда она доступна, используется параметр `CanExecute`. Он принимает имя метода или свойства, возвращающего `bool`:

```
[RelayCommand(CanExecute = nameof(CanSave))]
private async Task SaveAsync()
{
    // ...
}

public bool CanSave =>
    !string.IsNullOrWhiteSpace(Title) &&
```

```
    !string.IsNullOrWhiteSpace(Author) &&
    !IsSaving;
```

Кнопка «Сохранить» автоматически станет недоступной пока `CanSave` возвращает `false`. Осталось сообщить команде, что `CanSave` изменился — для этого добавим `[NotifyCanExecuteChangedFor]` к полям, от которых зависит `CanSave`:

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(CanSave))]
[NotifyCanExecuteChangedFor(nameof(SaveCommand))]
private string _title = "";

[ObservableProperty]
[NotifyPropertyChangedFor(nameof(CanSave))]
[NotifyCanExecuteChangedFor(nameof(SaveCommand))]
private string _author = "";

[ObservableProperty]
[NotifyPropertyChangedFor(nameof(CanSave))]
[NotifyCanExecuteChangedFor(nameof(SaveCommand))]
private bool _isSaving;
```

При изменении `_title`, `_author` или `_isSaving` — WPF переспросит `CanExecute` у `SaveCommand` и обновит доступность кнопки.

Параметр команды

Команде можно передать параметр через `CommandParameter` в XAML. Метод в этом случае принимает аргумент нужного типа:

```
[RelayCommand]
private void Delete(Book book)
{
    Books.Remove(book);
}
```

```
<Button Content="Удалить"
        Command="{Binding DeleteCommand}"
        CommandParameter="{Binding SelectedBook}" />
```

Полная ViewModel с командами

Соберём `BookEditViewModel` целиком — свойства из раздела 2 плюс команды:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
```

```
public partial class BookEditViewModel : ObservableObject
{
    private readonly IBookService _bookService;

    public BookEditViewModel(IBookService bookService)
        => _bookService = bookService;

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(CanSave))]
    [NotifyCanExecuteChangedFor(nameof(SaveCommand))]
    private string _title = "";

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(CanSave))]
    [NotifyCanExecuteChangedFor(nameof(SaveCommand))]
    private string _author = "";

    [ObservableProperty]
    private int _year = DateTime.Now.Year;

    [ObservableProperty]
    private string _genre = "";

    [ObservableProperty]
    private bool _isAvailable = true;

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(CanSave))]
    [NotifyCanExecuteChangedFor(nameof(SaveCommand))]
    private bool _isSaving;

    public bool CanSave =>
        !string.IsNullOrWhiteSpace(Title) &&
        !string.IsNullOrWhiteSpace(Author) &&
        !IsSaving;

    [RelayCommand(CanExecute = nameof(CanSave))]
    private async Task SaveAsync()
    {
        IsSaving = true;
        try
        {
            await _bookService.SaveAsync(new Book
            {
                Title  = Title,
                Author = Author,
                Year   = Year,
                Genre  = Genre
            });
```

```
        }
        finally
        {
            IsSaving = false;
        }
    }

    [RelayCommand]
    private void Cancel()
    {
        Title  = "";
        Author = "";
        Year   = DateTime.Now.Year;
        Genre  = "";
    }
}
```

Привязка команд в XAML

```
<Button Content="Сохранить"
        Command="{Binding SaveCommand}"
        Style="{StaticResource PrimaryButton}"
        IsDefault="True" />

<Button Content="Отмена"
        Command="{Binding CancelCommand}"
        Style="{StaticResource SecondaryButton}"
        IsCancel="True" />
```

Никакого `Click` в code-behind. Кнопка «Сохранить» недоступна пока `CanSave` возвращает `false`, блокируется на время выполнения запроса, `ProgressBar` появляется и исчезает через привязку к `IsSaving` — всё автоматически.

5. Работа с коллекциями

Большинство реальных экранов — это списки: список книг, список авторов, история заказов. В этом разделе разберём, как отображать коллекции в UI, реагировать на выбор элемента и управлять списком через команды.

ObservableCollection

Обычный `List<T>` не работает с привязками — он не реализует уведомления об изменениях. Добавили элемент в список — UI не узнает.

`ObservableCollection<T>` реализует `INotifyCollectionChanged` и автоматически уведомляет UI о любых изменениях: добавлении, удалении, перемещении элементов.

```
// Не работает с привязками — UI не обновится при Add/Remove
public List<Book> Books { get; } = new();

// Работает — UI обновляется автоматически
public ObservableCollection<Book> Books { get; } = new();
```

`ObservableCollection` объявляется как свойство только для чтения — саму коллекцию менять незачем, меняется её содержимое:

```
Books.Add(new Book { Title = "Мастер и Маргарита", Author = "Булгаков" });
Books.Remove(selectedBook);
Books.Clear();
```

ListBox — список с выбором элемента

`ListBox` отображает коллекцию и позволяет выбрать элемент. Привязывается через `ItemsSource` и `SelectedItem`:

```
<ListBox ItemsSource="{Binding Books}"
         SelectedItem="{Binding SelectedBook}" />
```

По умолчанию `ListBox` отображает результат `ToString()` каждого объекта. Чтобы управлять отображением, используется `DataTemplate`.

DataTemplate — шаблон элемента

`DataTemplate` описывает как отображается каждый элемент коллекции:

```
<ListBox ItemsSource="{Binding Books}"
         SelectedItem="{Binding SelectedBook}">
    <ListBox.ItemTemplate>
```

```
        <DataTemplate>
            <StackPanel Margin="4">
                <TextBlock Text="{Binding Title}"
                           FontWeight="SemiBold" />
                <TextBlock Text="{Binding Author}"
                           Foreground="Gray" FontSize="12" />
            </StackPanel>
        </DataTemplate>
    </ListBox.ItemTemplate>
</ListBox>
```

Внутри `DataTemplate` `DataContext` автоматически становится текущим элементом коллекции — поэтому `{Binding Title}` обращается к `Title` конкретной книги, а не всей ViewModel.

ListView — список с колонками

`ListView` с `GridView` отображает данные в виде таблицы — удобно для списков с несколькими полями:

```
<ListView ItemsSource="{Binding Books}"
          SelectedItem="{Binding SelectedBook}">
    <ListView.View>
        <GridView>
            <GridViewColumn Header="Название" Width="200"
                            DisplayMemberBinding="{Binding Title}" />
            <GridViewColumn Header="Автор"    Width="150"
                            DisplayMemberBinding="{Binding Author}" />
            <GridViewColumn Header="Год"      Width="60"
                            DisplayMemberBinding="{Binding Year}" />
        </GridView>
    </ListView.View>
</ListView>
```

`DisplayMemberBinding` — быстрый способ привязать текстовое значение к колонке без `DataTemplate`. Если нужно более сложное отображение — используем `CellTemplate`:

```
<GridViewColumn Header="Статус" Width="80">
    <GridViewColumn.CellTemplate>
        <DataTemplate>
            <TextBlock Text="{Binding IsAvailable, Converter={StaticResource
BoolToStatus}}"
                       Foreground="{Binding IsAvailable, Converter=
{StaticResource BoolToColor}}" />
        </DataTemplate>
    </GridViewColumn.CellTemplate>
</GridViewColumn>
```

SelectedItem — выбранный элемент

`SelectedItem` связывает выбранный элемент списка со свойством ViewModel. При клике на строку ViewModel узнаёт об этом автоматически. Объявляем `SelectedBook` через `[ObservableProperty]` и вешаем `[NotifyPropertyChangedFor]` для зависимого `IsBookSelected`:

```
[ObservableProperty]
[NotifyPropertyChangedFor(nameof(IsBookSelected))]
[NotifyCanExecuteChangedFor(nameof(EditCommand))]
[NotifyCanExecuteChangedFor(nameof(DeleteCommand))]
private Book? _selectedBook;

public bool IsBookSelected => SelectedBook != null;
```

При клике на строку `SelectedBook` обновится, `IsBookSelected` пересчитается, кнопки «Редактировать» и «Удалить» включатся автоматически по цепочке атрибутов.

BooksViewModel — полный пример

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

public partial class BooksViewModel : ObservableObject
{
    private readonly IBookService _bookService;

    public ObservableCollection<Book> Books { get; } = new();

    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(IsBookSelected))]
    [NotifyCanExecuteChangedFor(nameof(EditCommand))]
    [NotifyCanExecuteChangedFor(nameof(DeleteCommand))]
    private Book? _selectedBook;

    public bool IsBookSelected => SelectedBook != null;

    public BooksViewModel(IBookService bookService)
    {
        _bookService = bookService;
    }

    [RelayCommand]
    private void Add()
    {
        var view = new BookEditView
        {
            DataContext = new BookEditViewModel(_bookService)
```

```
        };

        if (view.ShowDialog() == true)
            Books.Add(view.DataContext is BookEditViewModel vm
                ? new Book { Title = vm.Title, Author = vm.Author }
                : new Book());
    }

    [RelayCommand(CanExecute = nameof(IsBookSelected))]
    private void Edit()
    {
        if (SelectedBook is null) return;

        var view = new BookEditView
        {
            DataContext = new BookEditViewModel(_bookService, SelectedBook)
        };

        view.ShowDialog();
    }

    [RelayCommand(CanExecute = nameof(IsBookSelected))]
    private async Task DeleteAsync()
    {
        if (SelectedBook is null) return;
        await _bookService.DeleteAsync(SelectedBook.Id);
        Books.Remove(SelectedBook);
    }

    [RelayCommand]
    private async Task LoadAsync()
    {
        var books = await _bookService.GetAllAsync();
        Books.Clear();
        foreach (var book in books)
            Books.Add(book);
    }
}
```

Полный пример: экран списка книг

```
<DockPanel>

    <!-- Шапка с кнопками -->
    <Border DockPanel.Dock="Top"
            BorderThickness="0,0,0,1" Padding="16,12">
        <DockPanel>
            <TextBlock DockPanel.Dock="Left"
                       Text="Каталог книг"
                       FontSize="18" FontWeight="SemiBold"
                       VerticalAlignment="Center" />
            <StackPanel DockPanel.Dock="Right"
                        Orientation="Horizontal">
                <Button Content="Обновить"
                        Command="{Binding LoadCommand}"
                        Style="{StaticResource SecondaryButton}" />
                <Button Content="+ Добавить"
                        Command="{Binding AddCommand}"
                        Style="{StaticResource PrimaryButton}"
                        Margin="8,0,0,0" />
            </StackPanel>
        </DockPanel>
    </Border>

    <!-- Панель кнопок для выбранной книги — видна только при выборе -->
    <Border DockPanel.Dock="Bottom"
            BorderThickness="0,1,0,0" Padding="16,8"
            Visibility="{Binding IsBookSelected,
                         Converter={StaticResource BoolToVis}}">
        <StackPanel Orientation="Horizontal">
            <Button Content="Редактировать"
                    Command="{Binding EditCommand}"
                    Style="{StaticResource SecondaryButton}" />
            <Button Content="Удалить"
                    Command="{Binding DeleteCommand}"
                    Style="{StaticResource DangerButton}"
                    Margin="8,0,0,0" />
        </StackPanel>
    </Border>

    <!-- Список -->
    <ListView ItemsSource="{Binding Books}"
              SelectedItem="{Binding SelectedBook}"
              BorderThickness="0">
        <ListView.View>
            <GridView>
                <GridViewColumn Header="Название" Width="250"
```

```
                                DisplayMemberBinding="{Binding Title}" />
                <GridViewColumn Header="Автор"    Width="180"
                                DisplayMemberBinding="{Binding Author}" />
                <GridViewColumn Header="Год"      Width="60"
                                DisplayMemberBinding="{Binding Year}" />
                <GridViewColumn Header="Жанр"     Width="120"
                                DisplayMemberBinding="{Binding Genre}" />
            </GridView>
        </ListView.View>
    </ListView>

</DockPanel>
```

При клике на строку `SelectedBook` обновляется, `IsBookSelected` становится `true`, панель с кнопками «Редактировать» и «Удалить» появляется автоматически — всё через привязки, без code-behind.

6. Навигация между экранами

Реальное приложение состоит из нескольких экранов. Нужно уметь переходить между ними, не нарушая архитектуру MVVM — то есть без создания окон прямо из ViewModel.

Два подхода к навигации

Несколько окон — каждый экран это отдельный `Window`. Просто в реализации, но окна живут независимо, сложно управлять состоянием и передавать данные между ними.

Один `ContentControl` — главное окно содержит один контейнер, в котором меняется текущий экран. Это стандартный подход для MVVM-приложений: вся навигация происходит через смену текущей ViewModel.

Мы будем использовать второй подход.

`ContentControl` — контейнер для текущего экрана

`ContentControl` отображает любой объект через `DataTemplate`. Если привязать его `Content` к текущей ViewModel — он автоматически покажет нужный View:

```
<!-- MainWindow.xaml -->
<DockPanel>

    <!-- Боковое меню -->
    <Border DockPanel.Dock="Left" Width="200"
            BorderThickness="0,0,1,0">
        <StackPanel Margin="8">
            <Button Content="Книги"
                    Command="{Binding NavigateToBooksCommand}" />
            <Button Content="Авторы"
                    Command="{Binding NavigateToAuthorsCommand}"
                    Margin="0,4,0,0" />
        </StackPanel>
    </Border>

    <!-- Текущий экран -->
    <ContentControl Content="{Binding CurrentViewModel}" />

</DockPanel>
```

`CurrentViewModel` — свойство в `MainWindowViewModel`, которое хранит активную ViewModel. При его смене `ContentControl` автоматически обновит отображаемый экран.

DataTemplate для сопоставления ViewModel → View

WPF не знает, какой View показать для каждой ViewModel. Нужно явно указать это через `DataTemplate` в ресурсах `App.xaml`:

```
<!-- App.xaml -->
<Application xmlns:vm="clr-namespace:LibraryApp.ViewModels"
             xmlns:views="clr-namespace:LibraryApp.Views">
    <Application.Resources>
        <DataTemplate DataType="{x:Type vm:BooksViewModel}">
            <views:BooksView />
        </DataTemplate>

        <DataTemplate DataType="{x:Type vm:AuthorsViewModel}">
            <views:AuthorsView />
        </DataTemplate>

        <DataTemplate DataType="{x:Type vm:BookEditViewModel}">
            <views:BookEditView />
        </DataTemplate>
    </Application.Resources>
</Application>
```

UserControl

При навигации через `ContentControl` каждый экран обычно оформляется как `UserControl`. Это просто переиспользуемая UI-обёртка для одного экрана: в ней лежит только разметка, а логика — во ViewModel.

Короткий пример:

```
<!-- Views/BooksView.xaml -->
<UserControl x:Class="LibraryApp.Views.BooksView"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Grid Margin="16">
        <TextBlock Text="Список книг"
                   FontSize="24"
                   FontWeight="SemiBold" />
    </Grid>
</UserControl>
```

```
// BooksView.xaml.cs
public partial class BooksView : UserControl
{
```

```
    public BooksView()
    {
        InitializeComponent();
    }
}
```

То есть `MainWindow` содержит только контейнер, а `BooksView`, `AuthorsView`, `BookEditView` и так далее подставляются внутрь этого контейнера через `DataTemplate`.

NavigationService

ViewModel не должна создавать окна или переключать UI напрямую — это нарушает разделение слоёв. Вместо этого навигация делегируется сервису.

```
// Services/INavigationService.cs
public interface INavigationService
{
    void NavigateTo<TViewModel>() where TViewModel : ObservableObject,
new();
    void NavigateTo(ObservableObject viewModel);
}
```

```
// Services/NavigationService.cs
public class NavigationService : INavigationService
{
    private readonly MainWindowViewModel _mainVm;

    public NavigationService(MainWindowViewModel mainVm)
        => _mainVm = mainVm;

    public void NavigateTo<TViewModel>() where TViewModel :
ObservableObject, new()
        => _mainVm.CurrentViewModel = new TViewModel();

    public void NavigateTo(ObservableObject viewModel)
        => _mainVm.CurrentViewModel = viewModel;
}
```

`MainWindowViewModel` хранит текущий экран и уведомляет UI при его смене:

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

public partial class MainWindowViewModel : ObservableObject
{
    private INavigationService? _navigation;
```

```
    [ObservableProperty]
    private ObservableObject? currentViewModel;

    public void Initialize(INavigationService navigation)
        => _navigation = navigation;

    [RelayCommand]
    private void NavigateToBooks()
        => Navigation.NavigateTo<BooksViewModel>();

    [RelayCommand]
    private void NavigateToAuthors()
        => Navigation.NavigateTo<AuthorsViewModel>();

    private INavigationService Navigation
        => _navigation ?? throw new
InvalidOperationException("NavigationService is not initialized.");
}
```

`[RelayCommand]` создаст свойства `NavigateToBooksCommand` и `NavigateToAuthorsCommand` автоматически, а `[ObservableProperty]` сгенерирует уведомляемое свойство `CurrentViewModel`.

Инициализация в `App.xaml.cs`

Здесь по-прежнему есть циклическая зависимость: `NavigationService` зависит от `MainWindowViewModel`, а `MainWindowViewModel` должен использовать `INavigationService`. Разрешаем её через отдельную инициализацию после создания объектов:

```
protected override void OnStartup(StartupEventArgs e)
{
    base.OnStartup(e);

    var mainVm = new MainWindowViewModel();
    var navigation = new NavigationService(mainVm);

    mainVm.Initialize(navigation);

    navigation.NavigateTo<BooksViewModel>();

    new MainWindow { DataContext = mainVm }.Show();
}
```

Так же нужно будет убрать `StartupUri` из `App.xaml`, так как теперь мы определили окно, которое будет открываться при запуске в code-behind.

Передача данных между экранами

Когда нужно передать данные на следующий экран — передаём их через конструктор ViewModel:

```
[RelayCommand]
private void Edit()
{
    if (SelectedBook is null)
        return;

    Navigation.NavigateTo(new BookEditViewModel(_bookService,
SelectedBook));
}
```

Возврат с результатом сложнее — ViewModel не знает о предыдущем экране. Простое решение: передать функцию callback:

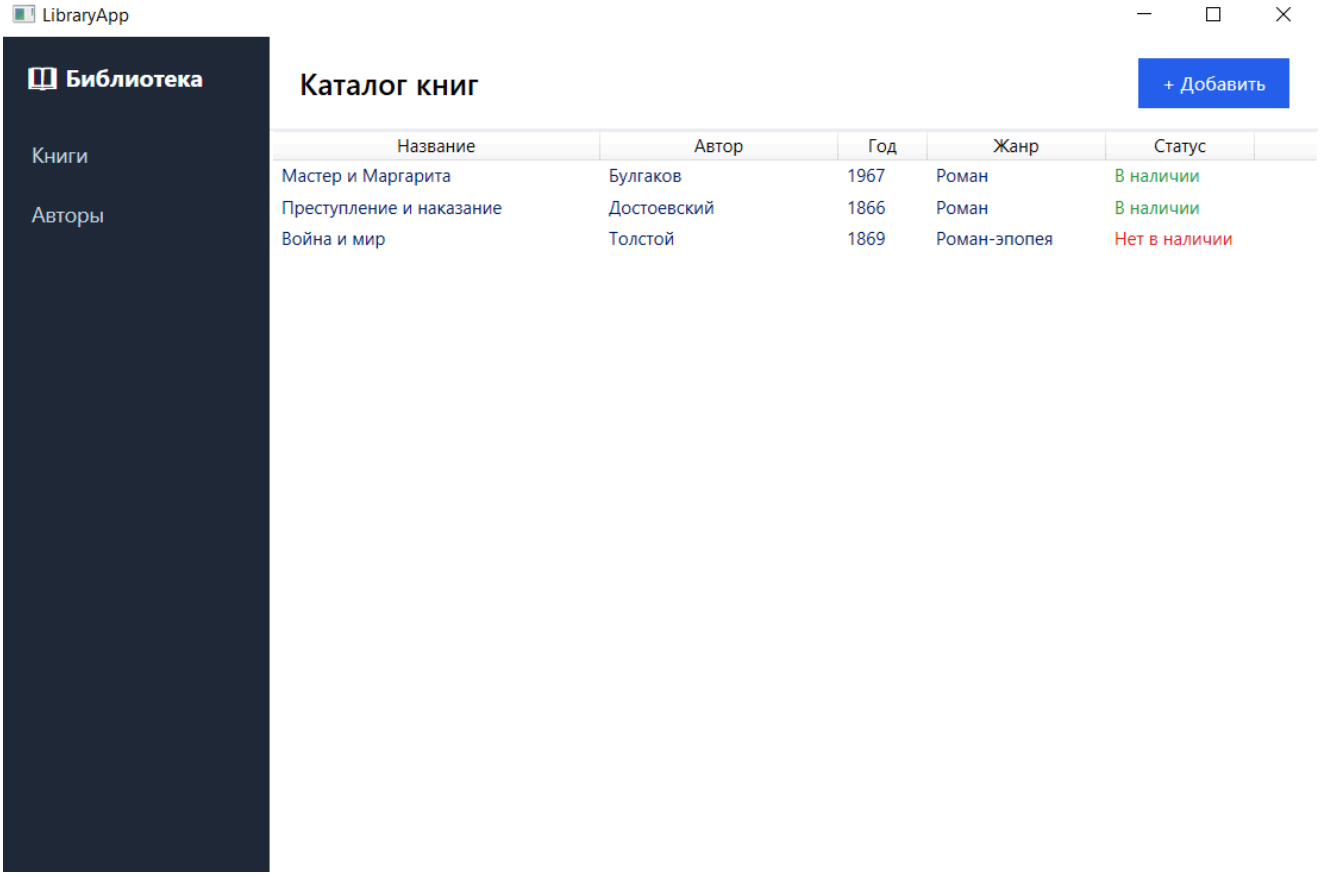
```
Navigation.NavigateTo(new BookEditViewModel(
    _bookService,
    SelectedBook,
    onSaved: savedBook =>
    {
        var existing = Books.FirstOrDefault(b => b.Id == savedBook.Id);
        if (existing != null)
            Books[Books.IndexOf(existing)] = savedBook;

        Navigation.NavigateTo<BooksViewModel>();
    }));
```

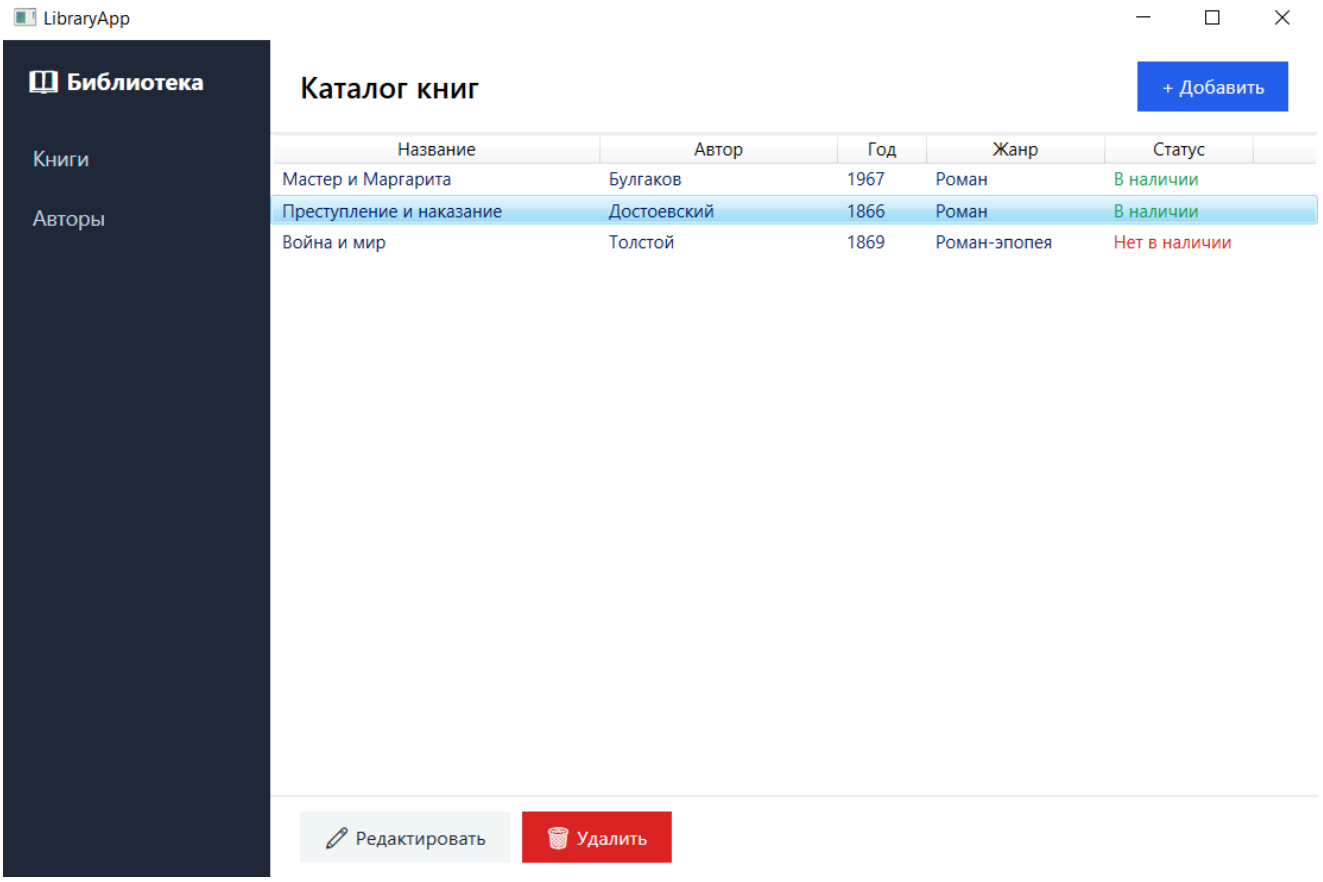
```
public partial class BookEditViewModel : ViewModelBase
{
    private readonly Action<Book>? _onSaved;

    public BookEditViewModel(
        IBookService service,
        Book book,
        Action<Book>? onSaved = null)
    {
        _onSaved = onSaved;

        // ...
    }

    [RelayCommand]
    private async Task SaveAsync()
```

```
    {
        var saved = await _service.SaveAsync(ToBook());
        _onSaved?.Invoke(saved);
    }
}
```

Пример приложения на WPF

В папке `Примеры` в учебном репозитории находится пример реализованного desktop-приложения `LibraryApp`, на примере которого мы изучали WPF в этих двух лекциях.





LibraryApp

Библиотека

Книги

Авторы

Редактировать книгу

Название: Преступление и наказание

Автор: Достоевский Ф.М.

Год: 1866

Жанр: Роман

☑ В наличии

Отмена | Сохранить

LibraryApp

Библиотека

Книги

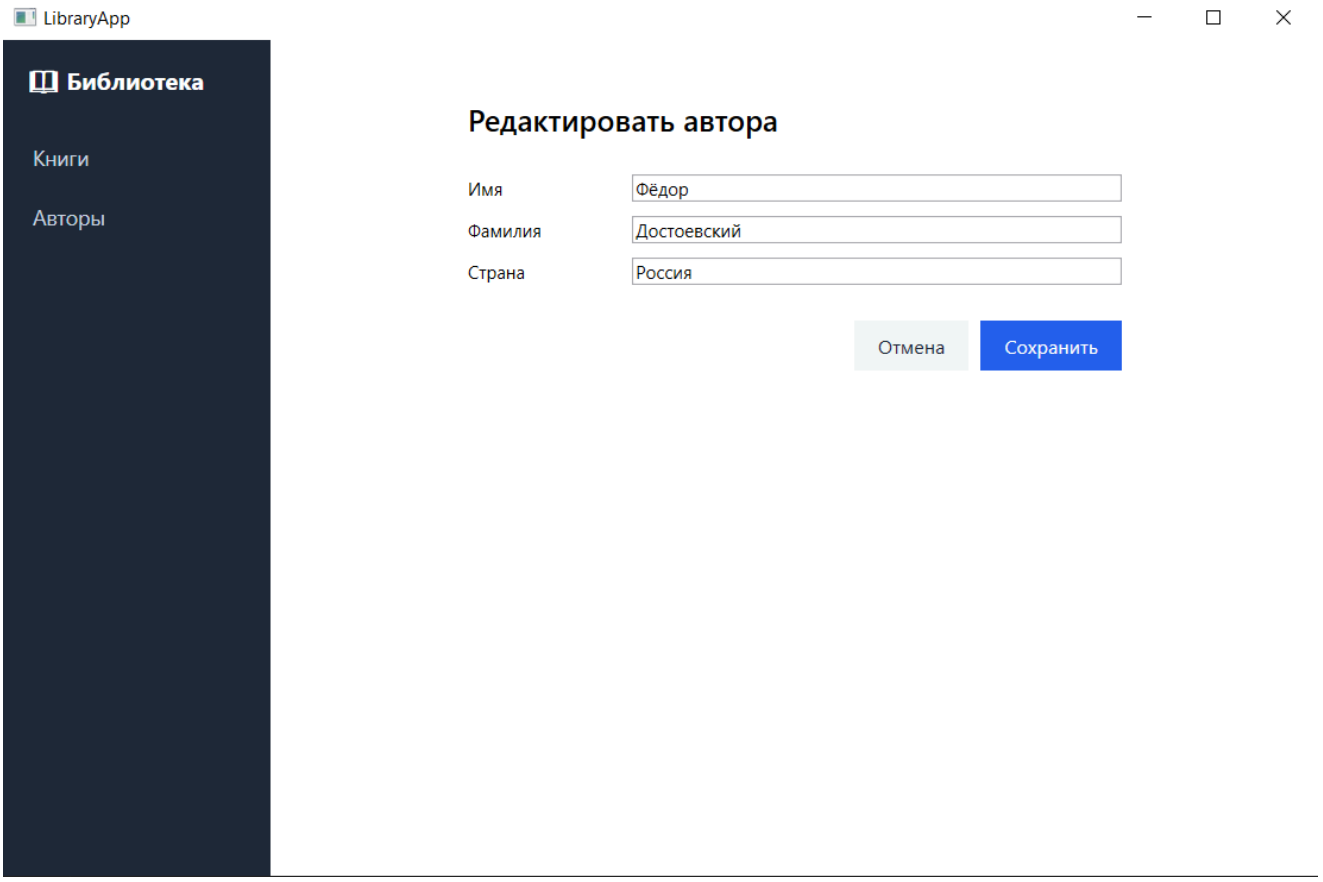
Авторы

Авторы

+ Добавить

Имя	Фамилия	Страна
Михаил	Булгаков	Россия
Фёдор	Достоевский	Россия
Лев	Толстой	Россия

Редактировать | Удалить


LibraryApp
Библиотека
Книги
Авторы
Редактировать автора
Имя
Фёдор
Фамилия
Достоевский
Страна
Россия
Отмена
Сохранить

Практическая задание

Цель

Разработать настольное приложение с графическим интерфейсом на WPF с тем же функционалом, что и консольная версия. Проект разбить на несколько слоёв в виде отдельных csproj-проектов внутри одного Solution.

Структура Solution

```
TodoApp.sln
├── TodoApp.Models/          ← Class Library
│   ├── TodoItem.cs
│   ├── TodoStatus.cs
│   ├── Profile.cs
│   └── TodoApp.Models.csproj
│
├── TodoApp.Data/            ← Class Library
│   ├── AppDbContext.cs
│   ├── TodoRepository.cs
│   ├── ProfileRepository.cs
│   ├── Migrations/
│   └── TodoApp.Data.csproj
│
└── TodoApp.Desktop/         ← WPF Application
    ├── Views/
    ├── ViewModels/
    ├── Services/
    ├── App.xaml
    └── TodoApp.Desktop.csproj
```

Зависимости между проектами:

- `TodoApp.Models` — не зависит ни от чего
- `TodoApp.Data` — зависит от `TodoApp.Models`, содержит EF Core пакеты
- `TodoApp.Desktop` — зависит от `TodoApp.Models` и `TodoApp.Data`

Что нужно сделать

1. Создать проекты и Solution

2. Перенести классы из консольного проекта

- `TodoItem`, `TodoStatus`, `Profiles` → в `TodoApp.Models`
- `AppDbContext`, `TodoRepository`, `ProfilePepository`, `Migrations/` → в `TodoApp.Data`

3. Разработать MVVM-архитектуру в Desktop-проекте

- `MainViewModel` — главная ViewModel
- `LoginViewModel` — ViewModel для входа в профиль
- `RegisterViewModel` — ViewModel для регистрации в профиле
- `TodoListViewModel` — ViewModel для просмотра списка задач и поиска
- `AddTaskViewModel` — ViewModel для формы добавления задачи
- `EditTaskViewModel` — ViewModel для формы редактирования задачи

4. Разработать интерфейс

Минимальный набор экранов:

- Главный экран — список всех задач с возможностью поиска и фильтрации по параметрам
- Форма регистрации
- Форма входа в существующий профиль
- Форма добавления задачи
- Форма редактирования задачи

Минимальный функционал (соответствует консольной версии):

- Просмотр списка задач с отображением статуса и даты
- Добавление новой задачи
- Редактирование текста задачи
- Удаление задачи
- Изменение статуса задачи (`NotStarted` / `InProgress` / `Completed` / `Postponed` / `Failed`)

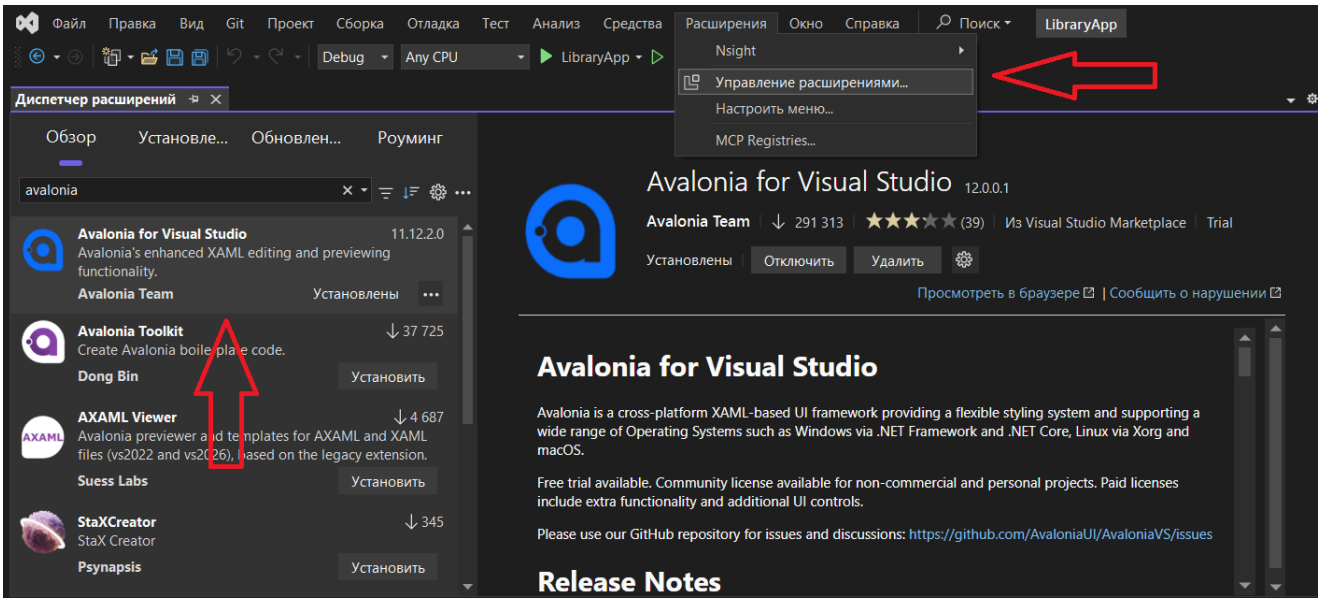
5. Связать ViewModel и репозитории

`MainViewModel` при инициализации загружает задачи из базы через `TodoRepository` и `ProfileRepository` . Каждая команда (добавить, удалить, изменить статус) сохраняет изменение в базу и обновляет коллекцию.

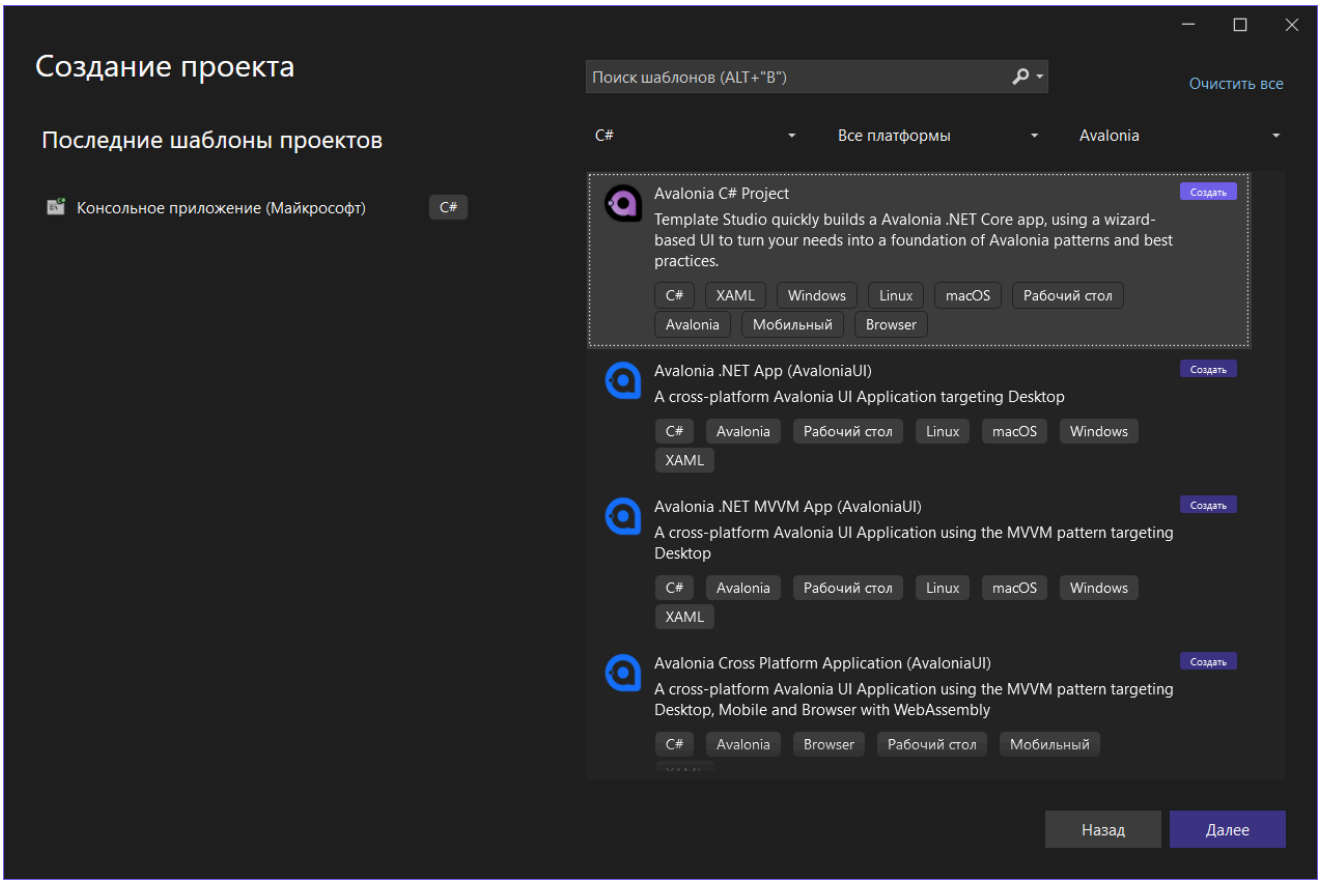
Avalonia UI

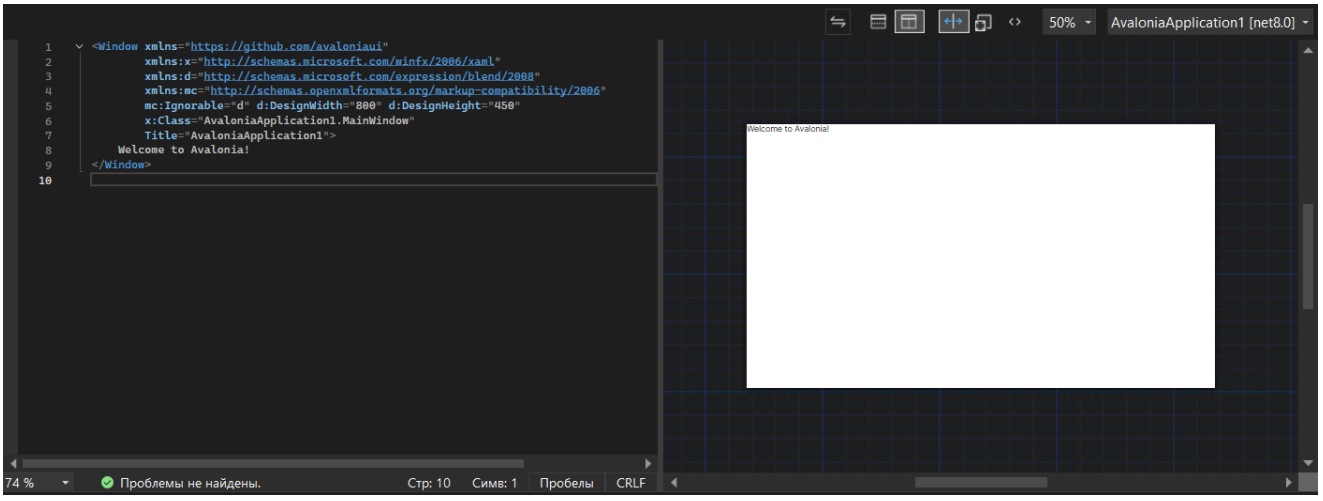
В качестве альтернативы (например если вы работаете на Linux или macOS) можно сделать это задание с использованием Avalonia — кроссплатформенного open-source фреймворка, похожего на WPF, с поддержкой Windows, Linux и macOS. Документацию по фреймворку можно найти на сайте [Avalonia UI - The Cross-Platform .NET UI Framework](#).

Для того чтобы работать с Avalonia в Visual Studio, нужно установить соответствующие расширение:



После этого вам станут доступны готовые шаблоны приложений на Avalonia и удобный интерфейс для разработки настольных приложений, аналогичный WPF:


Создание проекта
Последние шаблоны проектов
Консольное приложение (Майкрософт)
C#
Поиск шаблонов (ALT+"B")
Очистить все
C#
Все платформы
Avalonia
Avalonia C# Project
Создать
Template Studio quickly builds a Avalonia .NET Core app, using a wizard-based UI to turn your needs into a foundation of Avalonia patterns and best practices.
C# XAML Windows Linux macOS Рабочий стол Avalonia Мобильный Browser
Avalonia .NET App (AvaloniaUI)
A cross-platform Avalonia UI Application targeting Desktop
C# Avalonia Рабочий стол Linux macOS Windows XAML
Avalonia .NET MVVM App (AvaloniaUI)
A cross-platform Avalonia UI Application using the MVVM pattern targeting Desktop
C# Avalonia Рабочий стол Linux macOS Windows XAML
Avalonia Cross Platform Application (AvaloniaUI)
A cross-platform Avalonia UI Application using the MVVM pattern targeting Desktop, Mobile and Browser with WebAssembly
C# Avalonia Browser Рабочий стол Мобильный
Назад
Далее


50%
AvaloniaApplication1 [net8.0]
<Window xmlns="https://github.com/avaloniaui"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="450"
x:Class="AvaloniaApplication1.MainWindow"
Title="AvaloniaApplication1">
Welcome to Avalonia!
</Window>
Welcome to Avalonia!
74 %
Проблемы не найдены.
Стр: 10
Симв: 1
Пробелы
CRLF